# STRUCTURE OF COMPUTER SYSTEMS

## Approximating the Sine Function on FPGA

IEEE 754 Single-Precision, Pipelined AXI4-Stream Implementation

Student: Diaconu Mihai
Supervising professor: Lazea Dragos Bogdan

Technical University of Cluj-Napoca
Faculty of Automation and Computer Science

November 5, 2026

# Contents


**1 Introduction** **2**
- 1.1 Problem description . . . 2
- 1.2 Motivation . . . 2
- 1.3 Objectives . . . 3
- 1.4 Work plan . . . 3

**2 Bibliographic Study** **4**
- 2.1 Theoretical background . . . 4
  - 2.1.1 IEEE 754 single-precision floating point . . . 4
  - 2.1.2 Elementary function evaluation . . . 5
  - 2.1.3 Range reduction . . . 5
  - 2.1.4 AXI4-Stream interface . . . 5
- 2.2 Existing approaches for approximations . . . 7
  - 2.2.1 Table-based methods . . . 7
  - 2.2.2 Shift-and-add algorithms (CORDIC) . . . 7
  - 2.2.3 Polynomial approximation . . . 8
  - 2.2.4 Bit-manipulation techniques . . . 9
  - 2.2.5 Vivado IP cores . . . 10

**3 Analysis** **11**
- 3.1 Classification of binary32 inputs . . . 11
- 3.2 Range reduction for binary32 inputs . . . 11
- 3.3 CORDIC rotation for sine evaluation . . . 12
- 3.4 Lookup table with linear interpolation . . . 15
- 3.5 Normalization and rounding to binary32 . . . 17
- 3.6 Pipelined execution . . . 18
- 3.7 Flow control with backpressure . . . 19

**4 Design** **20**
- 4.1 Top-level architecture . . . 20
- 4.2 Range reduction unit . . . 21
- 4.3 The CORDIC core . . . 21
- 4.4 The Lookup Table Core . . . 23
- 4.5 Reconstruction and Normalization unit . . . 24

# Chapter 1

# Introduction

## 1.1 Problem description

The evaluation of non-linear elementary functions, in particular trigonometric functions such as the sine function, $\sin(x)$, is a frequent operation in digital systems. Applications such as machine learning and digital signal processing can often make do with fast, approximate evaluations of these functions [1]. This project considers real-valued inputs represented in the 32-bit single-precision (binary32) floating-point format defined by the IEEE 754 standard [2].

## 1.2 Motivation

Elementary functions such as $\sin(x)$ cannot be computed exactly with a finite number of arithmetic operations. Any implementation, either software or hardware, can only approximate them [1]. The precision of the result therefore becomes a design decision. General-purpose implementations aim for results as close as possible to the exact value, while many applications can tolerate a less accurate result if it is obtained faster [1].

Choosing how to make this trade-off is not straightforward. In his review of approximation techniques, Muller [1] does not recommend a single method: the most suitable one depends on the function being evaluated, the required accuracy, the implementation technology and the memory available for tables. He also notes that the methods differ in versatility, accuracy and scalability, which makes a general comparison difficult.

On a Field-Programmable Gate Array (FPGA), this trade-off can be measured directly. Each approximation method has a concrete cost in look-up tables (LUTs), DSP slices, block RAM, latency and maximum clock frequency, and the best choice can change with the required precision. For example, Muller reports work in which CORDIC proved to be a good candidate for low-precision FPGA implementations of the arctangent function [1].

General guidance from the literature is therefore not enough to select a method for a specific platform and number format. A reliable choice requires the candidate methods to be implemented on the same device and measured under identical conditions, so that their accuracy and hardware cost can be compared directly, including against the solutions already offered by the FPGA vendor.

## 1.3 Objectives

The main objective of this project is to design, implement and evaluate two pipelined hardware architectures that compute the sine of IEEE 754 single-precision floating-point numbers on an FPGA, exposed through AXI4-Stream interfaces, and to compare their accuracy, resource usage and latency with the Xilinx IP-cores available in the Vivado design suite.

The methods will be described in synthesizable VHDL, so that they can be implemented on a real device and validated in hardware on a Digilent Nexys A7 development board, in addition to being verified in simulation.

**Secondary objectives:**

1. Study the existing methods for approximating elementary functions and select the two methods to be implemented: CORDIC and a table-based method.
2. Design a range reduction stage that maps any finite binary32 input to the interval supported by the approximation methods, and handles the special values ($\pm 0$, subnormals, $\pm\infty$, NaN).
3. Implement a pipelined CORDIC architecture for the sine function.
4. Implement a pipelined table-based architecture for the sine function.
5. Wrap both architectures in AXI4-Stream interfaces that respect the handshake rules.
6. Verify both designs in simulation against the software model, and measure their absolute and relative error over the binary32 input range.
7. Synthesize the designs together with the equivalent Vivado IP-cores and compare accuracy, resource usage (LUTs, flip-flops, DSP slices, block RAM), latency and maximum clock frequency.
8. Demonstrate the designs running on the Nexys A7 board.

## 1.4 Work plan

Table 1.1: Targets for each project meeting

| Meeting | Target |
|---|---|
| PM 1 | Pick the project and begin studying |
| PM 2 | Introduction and bibliographic study |
| PM 3 | Analysis; design of the CORDIC-based unit |
| PM 4 | Design of the table-based unit |
| PM 5 | Start implementing methods in hardware |
| PM 6 | Finish hardware implementation and start testing |
| PM 7 | Comparison with the Vivado IP cores; final presentation |

# Chapter 2

# Bibliographic Study

## 2.1 Theoretical background

### 2.1.1 IEEE 754 single-precision floating point

Before going into the methods for approximating the sine function, we need to understand the way real numbers are represented in memory using the IEEE 754 single-precision floating point standard [2].

The standard defines arithmetic formats, rounding rules, operations and exception handling [2]. For this project specifically we are interested in the specifics of **binary32**. As its name suggests, the **binary32** uses 32 bits to represent a real number:

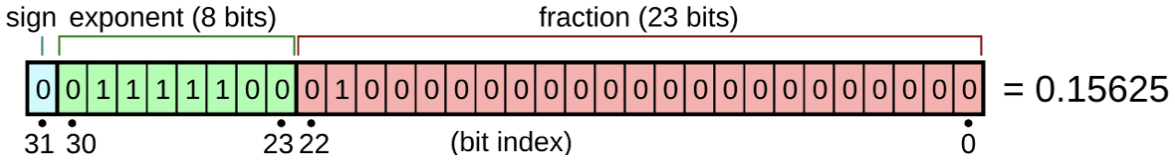



Figure 2.1: The IEEE 754 binary32 (single-precision) format [3]

The most significant bit gives the sign of the number — 0 for positive, 1 for negative values. Next 8 bits represent the exponent, biased with 127. The rest of 23 bits represent the mantissa (or fraction) which are the fractional part of the number with the implicit leading 1 bit for normalized values [1, 3]. The standard describes special exponents to represent special values like $\pm 0$, NaN, $\pm\infty$, as summarized in table 2.1 [3].

Table 2.1: Special values in the binary32 format

| Exponent $E$ | Fraction $f$ | Value |
|---|---|---|
| 0 | 0 | $\pm 0$ |
| 0 | $\neq 0$ | subnormal: $(-1)^s \times 2^{-126} \times 0.f$ |
| 255 | 0 | $\pm\infty$ |
| 255 | $\neq 0$ | NaN (not a number) |

For a normalized number, with sign bit $b_{31}$, biased exponent $E$ stored as an 8-bit unsigned integer and a 23-bit fraction, the represented value is

$$(-1)^{b_{31}} \times 2^{(b_{30}b_{29}\ldots b_{23})_2 - 127} \times (1.b_{22}b_{21}\ldots b_0)_2 \tag{2.1}$$

which can also be written as a sum over the fraction bits [1, 3]:

$$\text{value} = (-1)^{\text{sign}} \times 2^{(E-127)} \times \left(1 + \sum_{i=1}^{23} b_{23-i}\, 2^{-i}\right) \tag{2.2}$$

### 2.1.2 Elementary function evaluation

Following [1], the term *elementary functions* refers to the most widely used mathematical functions: sin, cos, tan, $\sin^{-1}$, $\cos^{-1}$, $\tan^{-1}$, sinh, cosh, tanh, $\sinh^{-1}$, $\cosh^{-1}$, $\tanh^{-1}$, as well as exponentials and logarithms in bases 2, $e$ and 10. These elementary functions can only be approximated, they cannot be computed exactly with a finite number of operations, as mentioned in section 1.2.

**Measuring accuracy.** Since an elementary function can only be approximated, the quality of an implementation is described by the error between the computed result $y^*$ and the exact result $y = f(x)$. Two measures are commonly used: the *absolute error*, $|y^* - y|$, and the *relative error*, $|y^* - y|/|y|$. The ideal behaviour is *correct rounding*, in which the implementation always returns the floating-point number nearest to the exact result [1].

### 2.1.3 Range reduction

Both CORDIC and the table-based method only work on a small input interval, while a binary32 input can be as large as about $3.4 \times 10^{38}$. Since sine is periodic, the input $x$ needs range reduction [4].

For large inputs, $\pi/2$ must be known to many more bits than the input precision, otherwise the error in $r$ becomes larger than $r$ itself; the Payne-Hanek algorithm addresses this problem [4].

### 2.1.4 AXI4-Stream interface

This subsection references the Structure of Computer Systems laboratory guide [5]. In a dataflow architecture, processing is organized as a pipeline of independent modules, where each module receives data from the previous one and passes its results to the next. Since modules may need a different number of clock cycles to process a data item, they rarely run at the same rate: a fast sender feeding a slower receiver must stall until the receiver can accept new data, so the throughput of the whole pipeline is bottlenecked by its slowest stage. Moreover, if a sender kept transmitting while its receiver was busy, data would be lost. A flow control mechanism is therefore needed between the stages.

The AXI4-Stream protocol is a great interface to connect modules that need to exchange data. It connects a sender and a receiver together relying on a **ready/valid** handshake principle.

The interface consists of 5 main signals: a shared `CLOCK`, a global `RESET` (active LOW), the `TDATA` lines, and 2 "control" bits: `TVALID` (from the sender to the receiver) and `TREADY` (from the receiver to the sender). Of course, the interface provides other optional signals, but the ones mentioned above are mandatory for reliable transfer of data between the modules.

Linking two modules together is simply done by connecting the output of the "sender" module to the inputs of the "receiver" module, but making sure `TREADY` goes from the receiver to the sender.

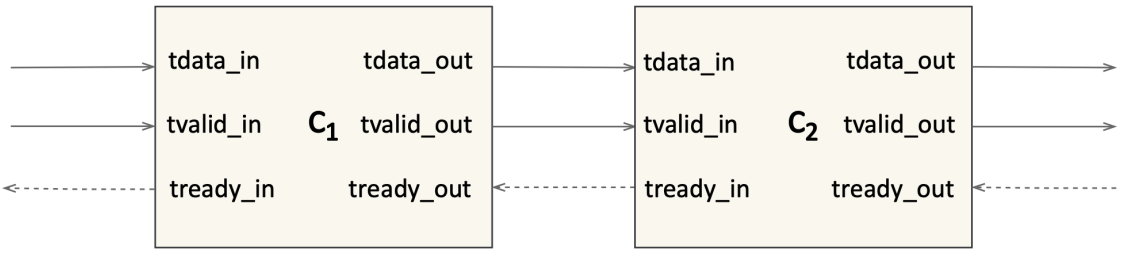



Figure 2.2: Two interconnected components using AXI4-Stream Interface [5]

**The core handshake rules.** As mentioned above, the `TVALID` and `TREADY` signals dictate when the information is transmitted across. This two-way mechanism allows both the sender and the receiver to control when the information is passed. The AXI4-Stream protocol imposes a very important rule, that is, for a data transfer to happen, both `TVALID` and `TREADY` signals must be asserted during the same rising edge.

For the protocol to work correctly, the sender and the receiver must respect four rules:

**Rule 1.** Once `TVALID` is asserted, it must remain asserted until the handshake occurs. This prevents the sender from "changing its mind" and losing the data.

**Rule 2.** The sender is not allowed to wait for `TREADY` before asserting `TVALID`. It must announce that it has new data even if the receiver is not yet ready to accept it.

**Rule 3.** The receiver is allowed to wait for `TVALID` before asserting `TREADY`, so it only signals readiness once valid data is available.

**Rule 4.** If the receiver asserts `TREADY` before `TVALID` has been asserted, it is allowed to deassert `TREADY` again and wait for the sender.

The following picture depicts the timing diagram for a simple data transfer:

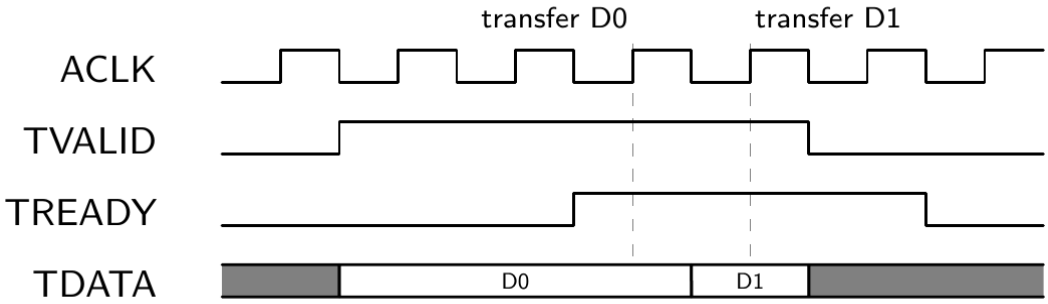



Figure 2.3: AXI4-Stream handshake: D0 is held until `TREADY` is asserted, then D0 and D1 are transferred on consecutive clock edges

## 2.2 Existing approaches for approximations

### 2.2.1 Table-based methods

Table-based methods go from pure tabulation of the results for as many inputs as possible when the needed precision is small, to more complex solutions that include lookup tables and some arithmetic operations (e.g. range reduction for periodic functions) [1].

When at most around 8 to 12 bits of accuracy are needed, the function can simply be tabulated, using the most significant bits of the input as the table address [1]. Following the notation of [1], let $f$ be the function to be approximated and let $x = 0.x_1x_2\ldots x_n$ be the input operand, represented in binary fixed-point arithmetic, where for simplicity $0 \leq x < 1$. The table is addressed by the first $p$ bits of $x$, so it holds $2^p$ entries. To simplify the notation, let $a = 0.x_1x_2\ldots x_p$ denote the value of the first $p$ bits of the input. Every input whose first $p$ bits are equal to those of $a$ lies in the interval

$$I_a = \left[\, a,\ a + 2^{-p} \right), \tag{2.3}$$

and all of these inputs read the same table entry $T_a$, so they all produce the same result. The question is therefore which value to store in each entry. The obvious choice is the value of the function at the left end of the interval,

$$T_a = f(a). \tag{2.4}$$

However, this is generally not optimal: since $f$ is usually monotonic over such a small interval, $f(x)$ is either always above or always below $f(a)$ inside $I_a$, so the error is entirely one-sided [1]. The absolute error is minimized instead by storing the midpoint between the smallest and the largest value of $f$ on the interval [1]:

$$T_a = \frac{1}{2}\left(\min_{x \in I_a} f(x) + \max_{x \in I_a} f(x)\right). \tag{2.5}$$

Some advantages of table-based methods are that they are faster than shift-and-add methods and also more accurate than bit manipulation techniques [1]. They are very versatile, since any continuous function can be tabulated. This approach is used both in hardware and software. The disadvantages are that they do not scale well. Table size grows exponentially with the number of input bits, so every extra bit of accuracy doubles the table. That is why simple tabulation only makes sense up to around 8–12 bits [1].

### 2.2.2 Shift-and-add algorithms (CORDIC)

As mentioned in [1], *shift-and-add* algorithms are among the oldest methods for evaluating elementary functions, going back to Henry Briggs (1561–1631), who used them to compute logarithms.

CORDIC (COordinate Rotation DIgital Computer) is a shift-and-add algorithm, introduced by Volder [6], that evaluates trigonometric functions using only additions, subtractions, bit shifts and a small table of precomputed constants, without requiring a hardware multiplier [1]. More generally, shift-and-add algorithms can also compute exponentials, logarithms and square roots [1].

The algorithm computes a rotation by an angle $\theta$ as a sequence of smaller rotations by fixed angles $\arctan 2^{-i}$, each performed in a direction $d_i = \pm 1$, which leads to the

iteration [1]

$$\begin{cases} x_{i+1} = x_i - d_i\, y_i\, 2^{-i} \\ y_{i+1} = y_i + d_i\, x_i\, 2^{-i} \\ z_{i+1} = z_i - d_i \arctan 2^{-i}. \end{cases} \tag{2.6}$$

Since the multiplications are only by $\pm 1$ and by powers of two, each iteration needs only additions, subtractions and shifts. The derivation of this iteration and its use for computing the sine function are detailed in section 3.3.

The main advantage of CORDIC is that it requires no hardware multipliers, only adders, shifters and a small table of constants [1], which makes it well suited for devices with few DSP slices. In rotation mode it also returns $\sin\theta$ and $\cos\theta$ at the same time, which simplifies the reconstruction of the result after range reduction (table 3.1). Moreover, since all iterations have the same structure, the algorithm maps naturally onto a pipeline, with one iteration per stage.

On the other hand, CORDIC converges linearly: each iteration adds roughly one bit of accuracy [1], so reaching full binary32 accuracy requires a large number of iterations, which increases both latency and area. To keep the accumulated rounding errors small, the internal datapaths must also be wider than the 24-bit significand of the result. Finally, the result is scaled by the gain $K$, which must be compensated either by a final multiplication or by starting from $x_0 = 1/K$.

### 2.2.3 Polynomial approximation

Polynomial approximation replaces an elementary function, such as $\sin x$, $e^x$ or $\ln x$, with a polynomial that can be evaluated using only additions and multiplications, which makes it a common choice in mathematical libraries [7].

**Weierstrass approximation theorem.** The Weierstrass approximation theorem states that any function $f$ continuous on a closed and bounded interval $[a, b]$ can be uniformly approximated by a polynomial to any desired accuracy [8]. The most direct way to build such a polynomial is interpolation: $f$ is evaluated at $n+1$ distinct points $x_0, x_1, \ldots, x_n$ in $[a, b]$, and the polynomial $p$ of degree at most $n$ is chosen such that $p(x_i) = f(x_i)$ for every $i$. This polynomial is given by the Lagrange interpolation formula [8]

$$p(x) = \sum_{i=0}^{n} f(x_i)\, L_i(x), \qquad L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}. \tag{2.7}$$

**Taylor series.** A Taylor series approximates $f$ around a point $x_0$ by a polynomial whose derivatives at $x_0$ match those of $f$:

$$p(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k. \tag{2.8}$$

For $x_0 = 0$ it is also called a Maclaurin series. For example, $\sin x \approx x - x^3/3! + x^5/5!$. The approximation is very accurate close to $x_0$, but its error grows quickly as $x$ moves away from the expansion point [8].

**The Remez algorithm.** To keep the error small over the whole interval, and not only near one point, the *minimax* polynomial is used instead: the polynomial of a given degree that minimizes the maximum absolute error over the interval. It is computed iteratively with the Remez algorithm [1].

The advantages of polynomial approximation are that there are few coefficients that need to be stored and accuracy increases with the degree of the polynomial. Also, Horner's scheme evaluates an $n$-degree polynomial with $n$ multiplications and $n$ additions.

On the other hand, these methods work only on a small interval, so they still require range reduction (section 3.2) and on an FPGA every multiplication costs DSP slices and adds latency.

### 2.2.4 Bit-manipulation techniques

Bit manipulation for elementary functions (such as $\log_2(x)$, $e^x$, $\sqrt{x}$, and $x^y$) relies on a powerful property of computer arithmetic: IEEE 754 floating-point representations store numbers logarithmically.

By abusing pointers or using bit-casts to treat a floating-point bit layout as a standard integer, you can compute incredibly fast, low-precision approximations of these complex mathematical functions using only basic integer shifts, additions, and subtractions [1].

As mentioned in section 2.1.1, the actual value of a **binary32** number is calculated as:

$$\mathrm{V} = (-1)^{\text{sign}} \times (1 + M) \times 2^{(E-127)} \tag{2.9}$$

If you bit-cast this exact 32-bit pattern directly, bit by bit, into a 32-bit integer, the resulting integer value is approximately a linear mapping of the base-2 logarithm of that number.

$$I_x \approx 2^{23} \left(\log_2 x + 127 - \sigma\right) \tag{2.10}$$

where $I_x$ is the binary32 encoding of $x > 0$ interpreted as a 32-bit unsigned integer, and $\sigma \approx 0.0450$ is a small correction constant [9].

Since $I_x$ is itself an approximation of $\log_2 x$, up to a scale factor and an offset, these functions can be approximated without iterations or polynomial evaluation.

The best-known application of this idea is the *fast inverse square root* routine from the source code of the video game *Quake III Arena* [9]. If $y = x^{-1/2}$, then $\log_2 y = -\frac{1}{2}\log_2 x$. Applying eq. (2.10) to both $x$ and $y$ gives

$$I_y \approx \frac{3}{2} \cdot 2^{23} \left(127 - \sigma\right) - \frac{1}{2} I_x, \tag{2.11}$$

so an approximation of $1/\sqrt{x}$ is obtained with only an integer shift and a subtraction. For $\sigma \approx 0.045$, the constant term is equal to `0x5F3759DF`, the "magic constant" found in the original code [1, 9].

For the sine function, no comparable approximation from the bit pattern exists, since $\sin x$ is periodic and is not related to the logarithm that the binary32 encoding approximates. Bit-level operations are nevertheless useful around the main approximation method: the identity $\sin(-x) = -\sin x$ is implemented by copying the sign bit, inputs with a small exponent can be returned unchanged since $\sin x \approx x$, and the special values of table 2.1 can be detected from the exponent and fraction fields alone. These techniques are therefore used in this project as part of the input and output stages, rather than as a standalone method.

### 2.2.5 Vivado IP cores

An intellectual property (IP) core is a reusable, pre-designed block of logic that can be integrated into an FPGA or ASIC design [5]. FPGA vendors provide libraries of such cores so that common functions do not have to be designed from scratch. In the Vivado design suite, AMD collects its IP cores in a single *IP catalog*, which covers areas such as embedded processing, communication and interfaces [5].

For the CORDIC algorithm presented in section 2.2.2, Xilinx provides the LogiCORE IP CORDIC core in the Vivado IP catalog [10]. The core operates on fixed-point two's complement data, with configurable input width, and exposes the AXI4-Stream interface. For sine and cosine, the input $\theta$ is given in radians in the range $[-\pi, \pi]$, and the core returns $\cos\theta$ and $\sin\theta$. Since it does not accept floating-point input, it also requires a range reduction stage and a conversion between floating point and fixed-point.

# Chapter 3

# Analysis

## 3.1 Classification of binary32 inputs

Not every input needs to go through the full pipeline: some have an obvious result, and some do not represent numbers at all. These inputs can be identified by examining only the exponent and fraction fields (table 2.1), before any computation takes place:

1. **Zero** (exponent and fraction both 0) is passed directly to the output, with its sign preserved, since the standard defines both $+0$ and $-0$ [2].
2. **Subnormal numbers** (exponent 0, fraction $\neq 0$) are smaller than $2^{-12}$, so $\sin x = x$ within the binary32 precision, and they are passed directly to the output.
3. **Tiny normal numbers**, with an exponent field smaller than 115 ($|x| < 2^{-12}$), are passed directly to the output for the same reason (see section 3.3).
4. **Small normal numbers**, with $|x| \leq \pi/4$, already lie in the reduced interval, so $k = 0$ and no quadrant correction is needed.
5. **All other normal numbers** go through range reduction (section 3.2) and the approximation method.
6. **Infinity** (exponent 255, fraction 0) produces NaN, since $\sin(\pm\infty)$ is an invalid operation [2].
7. **NaN** inputs (exponent 255, fraction $\neq 0$) produce NaN.

The inputs are therefore divided into two paths: a *bypass path*, for which the result is known immediately (cases 1–3, 6 and 7), and a *compute path*, which uses range reduction and the approximation method (cases 4 and 5).

## 3.2 Range reduction for binary32 inputs

Since sine is periodic, every input $x$ is first decomposed into an integer $k$ and a small remainder $r$ [4]:

$$x = k\,\frac{\pi}{2} + r, \qquad |r| \leq \frac{\pi}{4}, \quad k \in \mathbb{Z}, \tag{3.1}$$

where:

- $k$ is the number of quarter periods contained in $x$; the value $k \bmod 4$ determines the quadrant, and therefore which function and sign give the final result (table 3.1);
- $r$ is the *reduced argument*, which is passed to the approximation method (CORDIC or the lookup table).

Table 3.1: Reconstruction of $\sin x$ from the reduced argument $r$

| $k \bmod 4$ | $\sin x$ |
|---|---|
| 0 | $\sin r$ |
| 1 | $\cos r$ |
| 2 | $-\sin r$ |
| 3 | $-\cos r$ |

**Use of symmetry.** Since sine is an odd function, $\sin(-x) = -\sin x$, the reduction is performed on $|x|$, and the sign bit of the input is applied to the result at the very end.

**Multiplication by $2/\pi$.** A normalized binary32 input can be written as $x = M \cdot 2^E$, where $M$ is the 24-bit integer formed by the implicit leading bit and the fraction field, and $E = e - 150$ is derived from the stored exponent $e$, where $150 = 127 + 23$ accounts for the bias and for the 23 fraction bits that are shifted into the integer $M$. Computing $x \cdot \frac{2}{\pi}$ therefore reduces to multiplying the integer $M$ by the bits of $2/\pi$ and shifting the product by $E$ positions. Only a window of the bits of $2/\pi$ influences the result [4]: the bits that end up far above the binary point only add multiples of 4 to $k$, and since only $k \bmod 4$ is needed, they can be ignored. The bits that end up far below the binary point are smaller than the required precision, so they can be ignored as well. The constant $2/\pi$ is therefore stored once, with enough bits for the largest possible exponent, and the exponent $E$ selects the window of bits that is used for each input.

**Cancellation near multiples of $\pi/2$.** When $x$ is very close to a multiple of $\pi/2$, the reduced argument $r$ becomes very small. A good example is $x = 355$: since $355/113 \approx \pi$, the input is almost exactly $226 \cdot \frac{\pi}{2}$, which gives $k = 226$ and $r \approx 0.0000192$ quarter-turns. In binary, this value of $r$ starts with about 15 leading zeros, which, as in the case of small CORDIC results, carry no information. The window of $2/\pi$ must therefore be wide enough that $r$ still has 24 correct significant bits after these zeros, and this must hold for the worst-case binary32 input, the one closest to a multiple of $\pi/2$.

**Output of range reduction.** For each input $x$, this stage produces three values: the sign of the input, which is applied to the final result; the two bits of $k \bmod 4$, which select between $\pm\sin r$ and $\pm\cos r$ (table 3.1); and the reduced argument $r$, expressed in quarter-turns with $|r| \leq 0.5$, which is passed to the approximation method.

## 3.3 CORDIC rotation for sine evaluation

**Mathematical intuition.** To obtain the sine of an angle $\theta$, we can imagine rotating the unit vector $(1, 0)$ by $\theta$: the $y$-coordinate of the rotated vector is then equal to $\sin\theta$, and its $x$-coordinate to $\cos\theta$ (fig. 3.1).

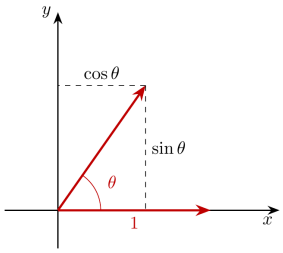



Figure 3.1: Rotating the unit vector $(1, 0)$ by an angle $\theta$ gives the point $(\cos\theta, \sin\theta)$

More generally, the coordinates of a vector $(x, y)$ rotated by $\theta$ are given by the rotation matrix:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \tag{3.2}$$

Factoring out $\cos\theta$, we obtain:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \cos\theta \begin{bmatrix} 1 & -\tan\theta \\ \tan\theta & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \tag{3.3}$$

The idea of CORDIC is to replace the rotation by $\theta$ with a sequence of smaller rotations by fixed angles $\theta_i$, each performed in a direction $d_i = \pm 1$, such that

$$\theta \approx \sum_{i=0}^{n-1} d_i \, \theta_i. \tag{3.4}$$

The angles are chosen such that $\tan\theta_i = 2^{-i}$, that is $\theta_i = \arctan 2^{-i}$, so that multiplying by $\tan\theta_i$ reduces to a right shift by $i$ positions. Since the same angles are used for every input, the values $\arctan 2^{-i}$ can be precomputed and stored in a table (table 3.2).

Table 3.2: Rotation angles $\theta_i = \arctan 2^{-i}$ used by CORDIC

| $i$ | $\tan\theta_i$ | $\theta_i$ (deg) | $\theta_i$ (rad) | $\theta_i$ (quarter-turns) |
|---|---|---|---|---|
| 0 | 1 | 45.00000000 | 0.7853981634 | 0.5000000000 |
| 1 | $2^{-1}$ | 26.56505118 | 0.4636476090 | 0.2951672353 |
| 2 | $2^{-2}$ | 14.03624347 | 0.2449786631 | 0.1559582608 |
| 3 | $2^{-3}$ | 7.12501635 | 0.1243549945 | 0.0791668483 |
| ... | ... | ... | ... | ... |
| 14 | $2^{-14}$ | 0.00349706 | 0.0000610352 | 0.0000388562 |
| 15 | $2^{-15}$ | 0.00174853 | 0.0000305176 | 0.0000194281 |

Applying eq. (3.3) for a single step $i$, with $\cos(d_i\theta_i) = \cos\theta_i$, gives:

$$\begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix} = \cos\theta_i \begin{bmatrix} 1 & -d_i \tan\theta_i \\ d_i \tan\theta_i & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} \tag{3.5}$$

After $n$ iterations, we have:

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = \left( \prod_{i=0}^{n-1} \cos\theta_i \right) \begin{bmatrix} 1 & -d_{n-1}\tan\theta_{n-1} \\ d_{n-1}\tan\theta_{n-1} & 1 \end{bmatrix} \cdots \begin{bmatrix} 1 & -d_0\tan\theta_0 \\ d_0\tan\theta_0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} \tag{3.6}$$

Since $\tan\theta_i = 2^{-i}$, the scale factor becomes

$$\prod_{i=0}^{n-1} \cos\theta_i = \prod_{i=0}^{n-1} \frac{1}{\sqrt{1+2^{-2i}}} \xrightarrow{n\to\infty} \frac{1}{K} \approx 0.6073, \tag{3.7}$$

where $K \approx 1.6468$ is the CORDIC gain [1]. This factor does not depend on the directions $d_i$, so it is the same for every input and can be compensated once, by starting from $x_0 = 1/K$ instead of 1.

Dropping the scale factor, and introducing a variable $z$ that keeps track of the angle still left to rotate, gives the CORDIC iteration presented in eq. (2.6) [1], where the values $\arctan 2^{-i}$ are read from table 3.2.

To compute $\sin\theta$, we use the *rotation* mode of CORDIC, in which $d_i = \text{sign}(z_i)$, so that $z$ is driven towards 0. Starting from $x_0 = 1/K$, $y_0 = 0$ and $z_0 = \theta$, after $n$ iterations we obtain $y_n \approx \sin\theta$ and $x_n \approx \cos\theta$.

**Convergence range.** CORDIC reaches the desired angle by adding or subtracting fixed angles: 45°, then 26.6°, then 14.0°, and so on. Even if all rotations are performed in the same direction, the largest angle that can be reached is

$$\sum_{i=0}^{\infty} \arctan 2^{-i} \approx 1.7433\,\text{rad} \approx 99.9°, \tag{3.8}$$

so the iteration converges only for $|z_0| \leq 1.7433$ rad [1]. After range reduction, the reduced argument satisfies $|r| \leq \pi/4 \approx 0.785$ rad (eq. (3.1)), which lies well inside this interval. Therefore, CORDIC converges for every input of the design, and no additional pre-rotation step is needed.

**Angle representation.** Range reduction (eq. (3.1)) multiplies the input by $2/\pi$, so its result is naturally expressed in *quarter-turns*, where one unit corresponds to $90° = \pi/2$ rad. The integer nearest to this product gives $k$, which selects the quadrant, while the difference between the product and $k$ gives the reduced argument $r$, which represents the position within the quadrant and satisfies $|r| \leq 0.5$. Converting $r$ back to radians would require an additional multiplication by $\pi/2$. Instead, $r$ is kept in quarter-turns, and the CORDIC rotation angles are stored in the same unit, as $\theta_i/(\pi/2)$ (table 3.2). The iteration in eq. (2.6) remains unchanged; only the stored constants differ.

**Sources of error.** The accuracy of the fixed-point CORDIC iteration is limited by three sources of error:

1. **The residual angle.** After $n$ iterations, the angle that remains to be rotated, $z_n$, is bounded by the last rotation angle, $\theta_{n-1} \approx 2^{-(n-1)}$, so each iteration adds roughly one correct bit to the result [1].

2. **Truncation in the shifts.** Each right shift by $i$ positions discards the bits that fall off the end of the word. Over many stages these losses accumulate, so the datapath needs extra guard bits beyond the required output precision.

3. **Rounding of the table constants.** Each value $\arctan 2^{-i}$ is stored with finite precision, which adds a small error at every stage.

**Precision of small results.** Inside the CORDIC iteration, values are represented in fixed point, so the position of the binary point never changes. A small result, such as $\sin r \approx 0.0002$, is therefore represented as

$$0.0002 = (0.\underbrace{000000000000}_{12\text{ zeros}}1101000110110111000101111\ldots)_2.$$

The leading zeros carry no information other than the magnitude of the number. When the result is converted to the binary32 format, normalization shifts the significand to the left until the first non-zero bit reaches the integer position, and the shift is recorded in the exponent:

$$0.0002 \approx (1.10100011011011100010111)_2 \times 2^{-13}.$$

The 12 leading zeros are discarded, and the 24 bits that follow them form the significand of the result. Since all of these bits must be correct, the fixed-point computation must be accurate down to bit position $12 + 24 = 36$, and not only to position 24. The same effect appears in decimal notation: obtaining three significant digits of 0.000123 requires computing six decimal places, since the first three are zeros.

**Required number of iterations and word length.** For a small reduced argument, $|r| < 2^{-12}$, the approximation $\sin r \approx r$ is used directly, since its relative error, approximately $r^2/6$, is below the binary32 precision. This covers both small inputs and inputs close to a multiple of $\pi/2$ (section 3.2). The smallest argument that reaches the CORDIC iteration is therefore about $2^{-12}$, which corresponds to the case illustrated above: the result has about 12 leading zeros, so an absolute accuracy of approximately $2^{-36}$ is required. Since each iteration adds roughly one correct bit, this requires about 36 to 38 iterations; with a small safety margin, the design uses about 40 iterations. In addition, the truncation errors of all stages accumulate, so a few guard bits must be added beyond the 36 bits required by the result, which leads to an internal word length of about 44 fractional bits.

## 3.4 Lookup table with linear interpolation

The second approximation method evaluates $\sin r$ from a precomputed table. This section determines how the table must be built and used to reach binary32 accuracy.

**Limits of direct tabulation.** A plain table stores the values of the sine function at $N$ evenly spaced points over one quadrant, $r \in [0, 1]$ quarter-turns, and every input that lies between two consecutive points receives the value of the point below it. The number of correct bits of this approximation grows only logarithmically with $N$: doubling the size of the table adds just one correct bit, even when the stored values are chosen optimally (eq. (2.5)) [1]. A table with 4096 entries reaches only about 11 correct bits, and a 24-bit result would require around 30 million entries.

**Linear interpolation.** Instead of using only the lower neighbour, both neighbours of the input can be used, and the sine function can be approximated by the straight line between them. If the input lies between the table points $i$ and $i + 1$, at a fraction $d$ of the distance between them ($0 \le d < 1$), then

$$\sin r \approx T[i] + d \cdot \big(T[i + 1] - T[i]\big), \tag{3.9}$$

where $T[i]$ is the stored value at point $i$. Each evaluation therefore requires two table reads, one multiplication and two additions.

**Required table size.** The error of linear interpolation is no longer proportional to the distance $h$ between table points, but to $h^2$, since the error of a first-order approximation grows with the square of the distance to the reference point [1]. Each doubling of the table now adds two correct bits instead of one. Table 3.3 compares the two approaches, as obtained by evaluating both methods numerically over the whole quadrant. With interpolation, 4096 entries are sufficient for about 25.7 correct bits, which exceeds the binary32 precision, while a plain table would need around 30 million entries. For the plain table, the absolute error is given, since its relative error near zero is unbounded. For linear interpolation, the errors in table 3.3 are relative, and they remain small even close to zero: since the table starts exactly at $\sin 0 = 0$, the line and the curve shrink together near the origin. Arguments smaller than $2^{-12}$ are handled directly, as described in section 3.3.

Table 3.3: Correct bits of the table-based approximation, with and without linear interpolation

| Table entries $N$ | Plain table | Linear interpolation |
|---|---|---|
| 256 | $\approx 7$ | $\approx 17$ |
| 1024 | $\approx 9$ | $\approx 21$ |
| 2048 | $\approx 10$ | $\approx 23$ |
| 4096 | $\approx 11$ | $\approx 25.7$ |

**One table for both sine and cosine.** As shown in table 3.1, range reduction requires either $\sin r$ or $\cos r$, depending on the quadrant. The CORDIC iteration provides both values at the same time, whereas a table-based method would in principle need two separate tables. However, with angles expressed in quarter-turns,

$$\cos\left(r \cdot \frac{\pi}{2}\right) = \sin\left((1 - r) \cdot \frac{\pi}{2}\right), \tag{3.10}$$

so the cosine is simply the sine read from the other end of the quadrant. A single table of the sine function over one quadrant therefore covers both cases: it is read at $|r|$ when the sine is needed, and at $1 - |r|$ when the cosine is needed. Since the argument can reach the end of the quadrant, the table stores $N + 1 = 4097$ values, including the endpoint $\sin(\pi/2) = 1$, which is also required for interpolating the last interval.

**Table addressing.** With $4096 = 2^{12}$ entries, the argument of the table, expressed in quarter-turns and lying in the interval $[0, 1]$, is divided directly by its bits:

$$(0.\underbrace{101100111010}_{\text{index } i}\ \underbrace{01101001\ldots}_{\text{fraction } d})_2.$$

The 12 most significant fractional bits select the table entry $i$, while the remaining bits form the fraction $d$, which gives the position of the argument between the entries $i$ and $i + 1$ (eq. (3.9)). Addressing the table therefore requires no arithmetic, only the selection of two bit fields. This is possible because the argument is expressed in quarter-turns: one quadrant corresponds exactly to the interval $[0, 1]$, so its binary fraction can be used as a table address directly.

## 3.5 Normalization and rounding to binary32

The outputs of both CORDIC and the table-based method are fixed-point numbers between 0 and 1, with about 44 fractional bits. Range reduction also provides the sign of the input and $k \bmod 4$. This stage will turn this number into a binary32 format in four steps.

**Sign of the result.** Two values determine the sign of the result: the sign of the input and the quadrant given by $k \bmod 4$. For $k \bmod 4 = 2$ or 3, i.e. in the third and fourth quadrants, the result is negative (table 3.1); this corresponds to the most significant bit of $k \bmod 4$ being 1. The sign of the result is therefore obtained as the exclusive OR of the input sign and this bit, since two negative factors cancel each other. Since the approximation method operates on $|r|$, the sign of $r$ is also carried as side information; it changes the sign of the result only when the sine is selected ($k \bmod 4 = 0$ or 2), since $\sin(-r) = -\sin r$ while $\cos(-r) = \cos r$.

**Leading-zero count.** The fixed-point result produced by the approximation method lies in the interval $[0, 1]$, and small results are preceded by a number of leading zeros. The next step is to count these zeros, which determines the position of the first non-zero bit and therefore the magnitude of the result. Let $z$ denote the number of leading zeros; the first non-zero bit is then at position $z + 1$.

**Normalization and exponent.** The fixed-point value is then shifted to the left by $z$ positions, so that the first non-zero bit reaches the position in front of the binary point. This bit becomes the implicit leading bit of the binary32 format, and only the bits that follow it are kept as the fraction. Since the value has the form $1.f \times 2^{-(z+1)}$, the stored exponent, including the bias of 127, is

$$e = 127 - (z + 1) = 126 - z. \tag{3.11}$$

**Rounding to 24 bits.** The significand of a binary32 number has only 24 bits: the implicit leading bit and the 23 fraction bits. The bits of the fixed-point result that follow these 24 bits must therefore be discarded, and the result is rounded according to the default rounding mode of the standard, *round to nearest, ties to even* [2]:

- if the first discarded bit is 0, the result is rounded down, i.e. the discarded bits are simply removed;
- if the first discarded bit is 1 and at least one of the following bits is 1, the result is rounded up, by adding one unit to the last kept bit;
- if the first discarded bit is 1 and all following bits are 0, the value lies exactly halfway between two candidates, and the one whose last bit is 0 is chosen, which avoids a systematic rounding bias.

Rounding up can cause an overflow of the significand: a value whose 24 bits are all 1, $(1.11\ldots1)_2$, becomes $(10.00\ldots0)_2$ after one unit is added to its last bit. In this case, the significand is shifted right by one position and the exponent $e$ (eq. (3.11)) is increased by one. This occurs, for instance, for results very close to 1.

It is worth mentioning that the value being rounded is already an approximation, so the result is not guaranteed to be correctly rounded. It can rarely differ by one unit in the last place, when the true value lies very close to a rounding boundary.

Bypassed inputs ($x$ itself, $\pm 0$, NaN) skip this stage, except the tiny-$r$ case, which goes through normalization.

## 3.6 Pipelined execution

Both approximation methods are organized as a pipeline, which accepts a new input at every clock cycle. CORDIC is particularly well suited to this organization: since its iterations all have the same structure, each iteration can be assigned to its own pipeline stage.

**Iterative vs pipelined execution.** An iterative algorithm with $n$ iterations can be executed in two ways: *iteratively*, by a single unit that gets reused for each iteration, or in a *pipeline*, by $n$ (mostly) identical units, each performing one iteration and separated by pipeline registers. Table 3.4 compares the two variants.

Table 3.4: Iterative and pipelined execution of an algorithm with $n$ iterations

| | Iterative | Pipelined |
|---|---|---|
| Iteration units | 1 | $n$ |
| Latency | $n$ cycles | $n$ cycles |
| New input accepted | every $n$ cycles | every cycle |
| Inputs processed at once | 1 | up to $n$ |

The latency is the same in both cases, since every input still needs $n$ iterations, but the throughput of the pipeline is $n$ times higher, and a new result is produced at each clock cycle. However, for the pipeline the price grows proportionally to the number of pipeline stages.

**Structure of one pipeline stage.** In a pipelined form, stage $i$ always does iteration $i$ of eq. (2.6), so the shift amount is fixed (stage 5 always shifts by 5) and the angle constant is fixed (stage 5 always uses $\arctan 2^{-5}$). In the iterative form, by contrast, the single unit is reused for every iteration, so it must shift by a different amount and use a different constant in each cycle, which requires a variable shifter and a table of constants. Each stage consists only of three additions or subtractions, plus a sign check of $z_i$. The fixed shifts require no computation, only a selection of the bits.

**Side information.** Each input carries extra information through the pipeline: the input sign, $k \bmod 4$, and the type of input (section 3.1). This information advances one stage per clock cycle together with the computed data so they arrive at the output at the same time. Bypassed inputs also must wait for the same number of cycles, otherwise the results would come out of order.

**The slowest stage sets the pace.** All stages share the same clock, so the clock can only be as fast as the slowest stage. The largest single operation of the design is the multiplication of the 24-bit significand with the selected window of $2/\pi$ in range reduction (section 3.2). If this operation cannot be completed within one clock cycle, it is divided over several pipeline stages, at the cost of a slightly higher latency.

**Analysing latency.** The CORDIC method has around 40 approximation stages (one per iteration), with 3 additions per stage and no multiplication, meaning that the total latency is relatively high — every input reaches the output after around 50 clock cycles, which include the range reduction, input classification and normalization after leaving the approximation stages. The cost of the approximation part is $40 \times 3$ adders.

The table-based method requires only a few approximation stages, which follow eq. (3.9): reading the entries $T[i]$ and $T[i+1]$, computing their difference, multiplying it by $d$ and adding the result to $T[i]$. Including the same preparation and normalization stages, its latency is around 10–12 clock cycles, and its approximation part requires one table of 4097 values and one multiplier.

The two chosen methods represent opposite trade-offs: CORDIC avoids multiplications and large memories at the cost of a longer pipeline, while the table-based method has a shorter pipeline, but requires a memory and a multiplier. Measuring the costs on the same device is one of the main objectives of this project (section 1.3).

## 3.7 Flow control with backpressure

The pipeline produces one result per clock cycle, but the receiver may refuse a result at any time through the valid/ready handshake (item 1). Since up to one input per stage is in flight, the pipeline must react without losing data. Each stage carries a valid bit that marks whether it holds a real input. Two strategies are possible: stalling all stages while the receiver is not ready, which needs no extra storage but requires a control signal that reaches every stage in the same cycle; or letting the pipeline run and collecting results in an output buffer, whose size must match the pipeline depth.

# Chapter 4

# Design

## 4.1 Top-level architecture

The top-level architecture of the CORDIC-based sine unit is shown in fig. 4.1. It consists of a **main path**, which computes the result for normal inputs, a **small-argument path**, used when the reduced argument is very small, a **bypass path**, which forwards the inputs with a known result directly to the output, delayed by the same number of clock cycles so that the order of the results is preserved, and the **side information** that accompanies each input through the pipeline (section 3.6).

Every input first passes through the input classification unit, which decides whether the input is computed or bypassed (section 3.1). On the main path, the range reduction unit follows, which produces the reduced argument $r$ and the value $k \bmod 4$ (section 3.2). After these two preparation units, the CORDIC core, consisting of 40 pipeline stages, computes $x_n \approx \cos r$ and $y_n \approx \sin r$ (section 3.3). The output of CORDIC is processed by the quadrant reconstruction unit, which either chooses between $\pm\sin$ and $\pm\cos$ using $k \bmod 4$, or takes $r \cdot \pi/2$ from the small argument path (table 3.1). Finally, we normalize and round the result (section 3.5) before passing it to the output of the pipeline. The output selector picks between the computed and the bypassed result.

The dotted red line represents the stall signal: when the last stage holds a valid result and the receiver is not ready, all stages hold their values and the unit stops accepting inputs. A global stall was chosen instead of an output buffer (section 3.7), since it requires no additional storage for the results in flight.

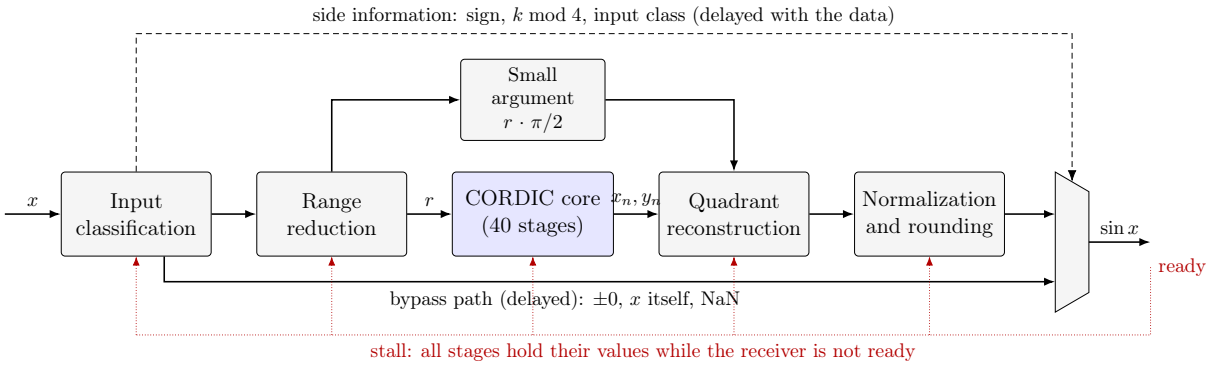



Figure 4.1: Top-level architecture of the CORDIC-based sine unit

## 4.2 Range reduction unit

The range reduction unit takes the binary32 fields of input $x$ (sign, exponent $e$, fraction $f$) and produces $|r|$, expressed in quarter turns, the sign of $|r|$, $k$ mod 4 and a "small argument" flag. The range reduction unit is the first part of the entire pipeline as the input classification part works on the same unpacked fields ($e$ and $f$). The pipeline of the unit is shown in fig. 4.2.

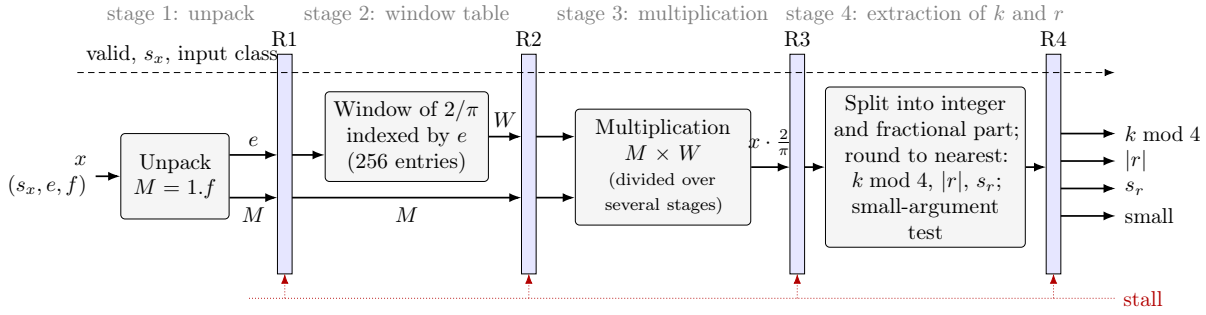



Figure 4.2: Pipeline of the range reduction unit

**Structure of the range reduction unit.** The unit consists of four stages:

**Stage 1.** The input is unpacked, and the significand $M = 1.f$ is built as a 24-bit integer (section 3.2).

**Stage 2.** The exponent selects a precomputed window $W$ of the bits of $2/\pi$. We store these values in a lookup-table with 256 entries instead of using a shifter [4].

**Stage 3.** We multiply $M \times W$ which gives $x \cdot 2/\pi$. This is the widest operation of the design, so it is going to be divided into more stages, as discussed in section 3.6.

**Stage 4.** The integer part of the multiplication gives $k$ mod 4. If the fraction is greater or equal to 0.5, $k$ is getting rounded up and $|r| = 1 - fraction$. The small argument test checks whether the first 12 fractional bits of $|r|$ are all zero.

The inputs with $|x| < \pi/4$ need no special case, since their product is below 0.5 and $k = 0$ comes out automatically.

## 4.3 The CORDIC core

The CORDIC pipeline is built out of 40 pipeline stages, as presented in fig. 4.3. Stage $i$ performs iteration $i$ of eq. (2.6). The stages differ only in their shift amount, $i$ positions, and in their constant $\alpha_i = \arctan(2^{-i})/(\pi/2)$, the rotation angle expressed in quarter-turns (table 3.2).

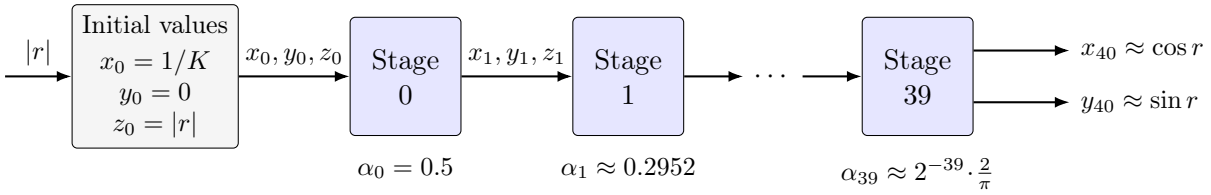



Figure 4.3: Chain of the 40 CORDIC pipeline stages

The starting values are initialized with $x_0 = 1/K$, $y_0 = 0$, $z_0 = |r|$, which are passed to the first pipeline stage. The sign of $r$ is carried as side information, so that all values in the core remain non-negative.

The output comes out of stage 39, where $x_{40}$ will store the approximation of $\cos r$ and $y_{40}$ will store the approximation of $\sin r$, in a fixed-point representation.

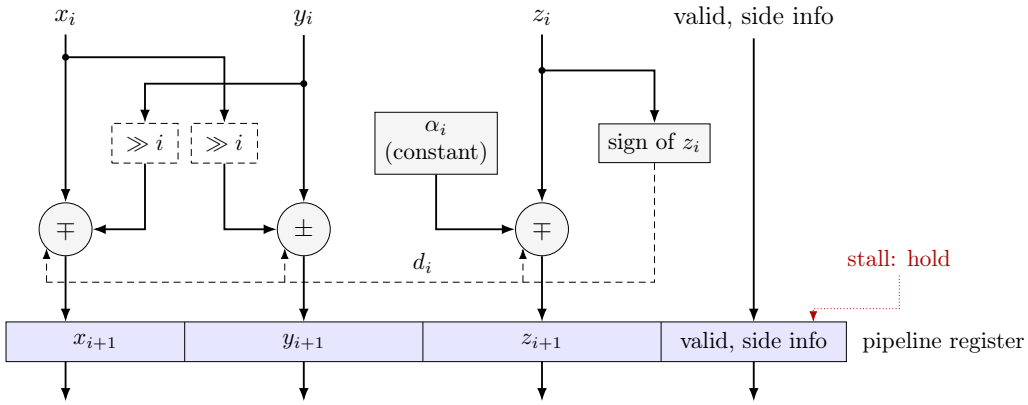



Figure 4.4: Structure of CORDIC pipeline stage $i$

**Structure of a pipeline stage.** The structure of stage $i$ is shown in fig. 4.4. It contains the following elements:

1. **Inputs:** the $(x_i, y_i, z_i)$ triplet, together with the valid bit and the side information, passed from the previous stage.

2. **Two fixed shifts:** $y_i \gg i$ is passed to the $x$ path and $x_i \gg i$ to the $y$ path. Since the shift amount of each stage is constant, the shifts require no logic, only a selection of bits. They are arithmetic shifts: the sign bit is replicated into the vacated positions, so that negative values remain correct.

3. **Direction detection:** the sign of $z_i$ determines the direction $d_i$, which selects for all three paths whether an addition or a subtraction is performed.

4. **Three add/subtract units:** they compute $x_{i+1} = x_i - d_i\,(y_i \gg i)$, $y_{i+1} = y_i + d_i\,(x_i \gg i)$ and $z_{i+1} = z_i - d_i\,\alpha_i$, where $\alpha_i$ is the constant of the stage.

5. **Pipeline register:** it stores $x_{i+1}$, $y_{i+1}$, $z_{i+1}$, the valid bit and the side information for the next stage, and keeps its contents unchanged while the stall signal is active (section 4.1).

The values $x$, $y$ and $z$ are represented as 46-bit two's-complement fixed-point numbers, with one sign bit, one integer bit and 44 fractional bits, following the word length determined in section 3.3.

## 4.4 The Lookup Table Core

The Lookup Table core goes into the same place as the CORDIC core in the top-level architecture (fig. 4.1). Its job is to take $|r|$ and the quadrant, and return either $\sin r$ or $\cos r$ using the table described in section 3.4.

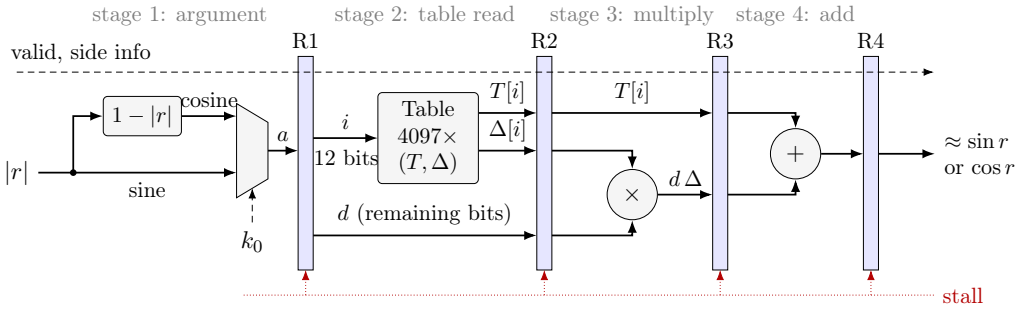



Figure 4.5: Pipeline of the table-based core

**Structure of the Lookup Table Pipeline.**

**Stage 1. Argument selection.** The input of the Lookup Table core are the $|r|$ and the quadrant's least significant bit – $k_0$ of $k \bmod 4$ – which decides if either the sine of cosine is needed, selecting between $|r|$ and $1 - |r|$ as the table address input $a$ (eq. (3.10)).

**Stage 2. Table read.** The output $a$ from the first stage is splits into two parts: the top 12 bits represent the index $i$, which selects the table entry $T[i]$ and the difference $\Delta[i] = T[i+1] - T[i]$, and the remaining bottom bits $d$ (passed to the next stage).

**Stage 3. Multiplication.** $d$ is multiplied with $\Delta[i]$ to get the value that needs to be added to $T[i]$ (which waits one stage to get to the adder at the same time as the multiplication result).

**Stage 4. Final addition.** Following eq. (3.9), we add $T[i]$ and the multiplication result $d \cdot \Delta[i]$ to get the final approximation of $\sin r$ or $\cos r$ as a fixed-point value to send to the normalization unit.

The table stores the values $T[i]$ with 40 fractional bits and the differences $\Delta[i]$ with 29 fractional bits, while the fraction $d$ has around 32 bits; these widths are required for the $2^{-36}$ accuracy of small results (section 3.3). The latency of the core is 4 clock cycles, compared with the 40 cycles of the CORDIC core, since the table-based method trades memory and a multiplier for a much shorter pipeline (section 3.6).

## 4.5 Reconstruction and Normalization unit

The reconstruction and normalization unit converts the fixed-point result of the approximation core into the finaly 32-bit floating point value. Since both approximation methods output a non-negative fixed-point value between 0 and 1, the same unit will be used for the CORDIC-based core and the table-based core. The inputs are the result of the core, the value $|r| \cdot \pi/2$ of the small argument path and the side information carried through the pipeline (the valid bit, $k$ mod 4, the signs of the original input $x$ and $r$, the small argument flag and the input class). The output is the final output, namely the binary32 result $\sin x$. The unit is organised into a five-stage pipeline shown in fig. 4.6.

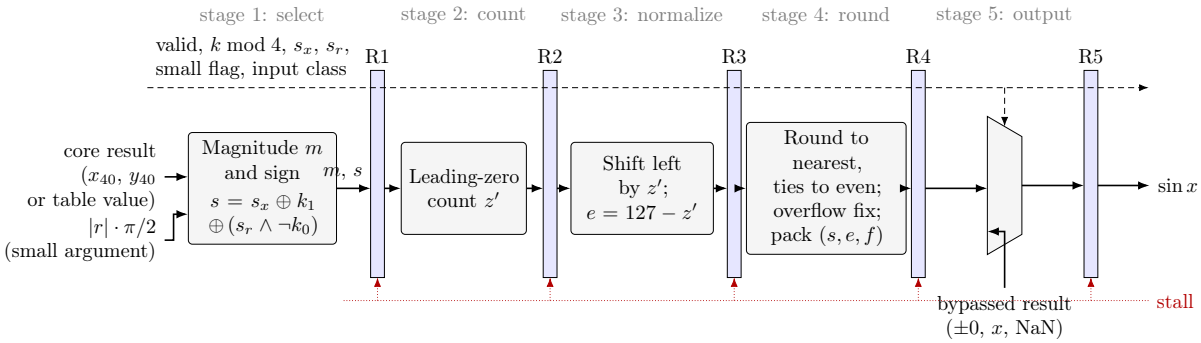



Figure 4.6: Pipeline of the reconstruction and normalization unit

**Structure of the reconstruction and normalization unit.**

**Stage 1.** Takes the inputs from the last stage of the core and the small argument. The magnitude $m$ comes from $x_{40}$ if $k_0 = 1$ (cosine) or $y_{40}$ if $k_0 = 0$ (sine) for CORDIC, from the table value if we use the LUT, or from the $|r| \cdot \pi/2$ when the small flag is set and we need the sine. The final sign of the output is $s = s_x \oplus k_1 \oplus (s_r \wedge \neg k_0)$ (section 3.5, Sign of the result).

**Stage 2.** Count the number of zeros $z'$, from the integer bit.

**Stage 3.** Shift left by $z'$, and set $e = 127 - z'$. Counting from the integer bit also handles $m = 1.0$ ($\cos 0$) and for $m < 1$ it equals eq. (3.11) ($z' = z + 1$).

**Stage 4.** Rounding to the nearest, then pack ($s$, $e$, $f$) (section 3.5).

**Stage 5.** Finally, the input class selects the computeed or the bypassed result. For NaN inputs and $\pm\infty$ we output a single NaN pattern.

# Bibliography


[1] J.-M. Muller, "Elementary functions and approximate computing," *Proceedings of the IEEE*, vol. 108, no. 12, pp. 2136–2149, 2020. DOI: 10.1109/JPROC.2020.2991885. HAL: hal-02517784v2.

[2] Wikipedia contributors, "IEEE 754." Wikipedia, The Free Encyclopedia, 2026. Accessed: Oct. 2026. Available: `https://en.wikipedia.org/wiki/IEEE_754`.

[3] Wikipedia contributors, "Single-precision floating-point format." Wikipedia, The Free Encyclopedia, 2026. Accessed: Oct. 2026. Available: `https://en.wikipedia.org/wiki/Single-precision_floating-point_format`.

[4] T. Ly, "A performance enhancement of the Payne-Hanek range reduction algorithm." arXiv preprint arXiv:2609.35015v2 [cs.MS], 2026. Available: `https://arxiv.org/abs/2609.35015`.

[5] Structure of Computer Systems, "Laboratory 4: Dataflow architectures. AXI4-Stream standard." Laboratory guide, Technical University of Cluj-Napoca, 2026.

[6] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Transactions on Electronic Computers*, vol. EC-8, pp. 330–334, Sept. 1959. DOI: 10.1109/TEC.1959.5222693.

[7] H. Becker, M. Tekriwal, E. Darulova, A. Volkova, and J.-B. Jeannin, "Dandelion: Certified approximations of elementary functions." arXiv preprint arXiv:2202.05472v2 [cs.PL], 2022. DOI: 10.48550/arXiv.2202.05472.

[8] J. Ylostalo, "Function approximation using polynomials," *IEEE Signal Processing Magazine*, vol. 23, no. 5, pp. 99–102, 2006.

[9] Wikipedia contributors, "Fast inverse square root." Wikipedia, The Free Encyclopedia, 2026. Available: `https://en.wikipedia.org/wiki/Fast_inverse_square_root`. Accessed: Oct. 19, 2026.

[10] Xilinx, *CORDIC v6.0 LogiCORE IP Product Guide (PG105)*. Xilinx, Inc., Aug. 2021. Available: `https://docs.amd.com/api/khub/documents/NHMqdvRJIfgF8hdbQmLFuA/content`.